

K L Deemed to be University

Department of Computer Science and Engineering | 2026-2027, Odd Semester
Database Systems Engineering and Distributed Backend Development (25CS1302E)

Skilling Session 5: MongoDB - Document Database Engineering

Name	Habeeb	Roll No.	2520090054
Course	25CS1302E	CO Mapping	CO2
Session	Skilling 5	Date	

Aim

To understand the BSON document model, collections and documents, choose between embedded and referenced document patterns, perform complete CRUD with PyMongo (Python) and Mongoose (Node.js), and query with comparison, logical, array, element and regex operators.

Program

crud_pymongo.py

```
"""
Skilling 5 (Part A): MongoDB document engineering with PyMongo - Task Hub.

Run: pip install pymongo
     python crud_pymongo.py
"""
import os
from datetime import datetime
from pprint import pprint

from pymongo import ASCENDING, MongoClient
from pymongo.errors import DuplicateKeyError

client = MongoClient(os.getenv("MONGO_URI", "mongodb://localhost:27017"))
db = client["taskhub_db"]
db.drop_collection("projects")
db.drop_collection("tasks")
db.drop_collection("users")

def show(title, cursor_or_doc):
    print(f"\n--- {title} ---")
    if isinstance(cursor_or_doc, dict) or cursor_or_doc is None:
        pprint(cursor_or_doc, sort_dicts=False)
    else:
        for doc in cursor_or_doc:
            pprint(doc, sort_dicts=False, width=110)

# ===== Embedded vs referenced patterns =====
# EMBEDDED: the checklist and comments are always read together with the task,
#           so they are stored inside the task document (one read, atomic update).
# REFERENCED: users are shared by many projects and tasks, so tasks store only
#           assignee_id (like a foreign key) and the user lives in its own collection.
db.users.create_index([("email", ASCENDING)], unique=True)
users = db.users.insert_many([
    {"_id": 1, "name": "Habeeb", "email": "habeeb@taskhub.dev", "skills": ["python", "sql", "fastapi"]},
    {"_id": 2, "name": "Ananya Rao", "email": "ananya@taskhub.dev", "skills": ["react", "css"]},
    {"_id": 3, "name": "Rahul Verma", "email": "rahul@taskhub.dev", "skills": ["java", "spring", "sql"]},
])
print("Inserted users:", users.inserted_ids)

# ===== CREATE =====
one = db.tasks.insert_one({
    "_id": 101,
    "title": "Design ER diagram",
    "project": "Task Hub Backend",
    "status": "DONE",
    "priority": 1,
    "estimate_hours": 6,
    "assignee_id": 1,
    "tags": ["db", "design"],
    "checklist": [
        # referenced
        # embedded
    ]
})
```

```

        {"item": "List entities", "done": True},
        {"item": "Add cardinalities", "done": True},
    ],
    "created_at": datetime(2026, 9, 18, 10, 0),
})
print("insert_one ->", one.inserted_id)

many = db.tasks.insert_many([
    {"_id": 102, "title": "Create PostgreSQL schema", "project": "Task Hub Backend",
     "status": "IN_PROGRESS", "priority": 1, "estimate_hours": 8, "assignee_id": 3,
     "tags": ["db", "sql"], "checklist": [{"item": "DDL", "done": True},
                                           {"item": "Indexes", "done": False}]},
    {"_id": 103, "title": "Write FastAPI CRUD", "project": "Task Hub Backend",
     "status": "TODO", "priority": 2, "estimate_hours": 12, "assignee_id": 1,
     "tags": ["api", "python"], "checklist": []},
    {"_id": 104, "title": "Build dashboard page", "project": "Task Hub Frontend",
     "status": "IN_PROGRESS", "priority": 2, "estimate_hours": 9, "assignee_id": 2,
     "tags": ["ui", "react"], "checklist": [{"item": "Charts", "done": False}]},
    {"_id": 105, "title": "Write release notes", "project": "Task Hub Frontend",
     "status": "TODO", "priority": 3, "assignee_id": None, "tags": ["docs"]},
])
print("insert_many ->", many.inserted_ids)

try:
    db.users.insert_one({"_id": 4, "name": "Copy", "email": "habeeb@taskhub.dev"})
except DuplicateKeyError:
    print("Duplicate email rejected by the unique index")

# ===== READ =====
show("find_one by _id", db.tasks.find_one({"_id": 101}, {"_id": 1, "title": 1, "checklist": 1}))
show("find all tasks of 'Task Hub Backend' (projection)",
     db.tasks.find({"project": "Task Hub Backend"}, {"_id": 0, "title": 1, "status": 1}))

# Comparison operators: $eq $ne $gt $gte $lt $lte $in $nin
show("$gte: estimate_hours >= 9",
     db.tasks.find({"estimate_hours": {"$gte": 9}}, {"_id": 0, "title": 1, "estimate_hours": 1}))
show("$in: status in TODO / IN_PROGRESS and $ne: priority != 1",
     db.tasks.find({"status": {"$in": ["TODO", "IN_PROGRESS"]}, "priority": {"$ne": 1}},
                   {"_id": 0, "title": 1, "status": 1, "priority": 1}))

# Logical operators: $and $or $nor $not
show("$or: priority 1 OR tagged 'ui'",
     db.tasks.find({"$or": [{"priority": 1}, {"tags": "ui"}]}, {"_id": 0, "title": 1}))
show("$and + $not: Backend tasks whose status is not DONE",
     db.tasks.find({"$and": [{"project": "Task Hub Backend"},
                             {"status": {"$not": {"$eq": "DONE"}}}]}, {"_id": 0, "title": 1, "status": 1}))
show("$nor: neither DONE nor priority 3",
     db.tasks.find({"$nor": [{"status": "DONE"}, {"priority": 3}]}, {"_id": 0, "title": 1}))

# Array operators: $all $size $elemMatch
show("$all: tagged both 'db' and 'sql'", db.tasks.find({"tags": {"$all": ["db", "sql"]}}, {"_id": 0, "title": 1}))
show("$size: exactly 2 checklist items", db.tasks.find({"checklist": {"$size": 2}}, {"_id": 0, "title": 1}))
show("$elemMatch: has an unfinished checklist item",
     db.tasks.find({"checklist": {"$elemMatch": {"done": False}}}, {"_id": 0, "title": 1}))

# Element operators: $exists $type
show("$exists: tasks without estimate_hours",
     db.tasks.find({"estimate_hours": {"$exists": False}}, {"_id": 0, "title": 1}))
show("$type: assignee_id is null (unassigned)",
     db.tasks.find({"assignee_id": {"$type": "null"}}, {"_id": 0, "title": 1}))

# Regex
show("$regex: title starts with 'Write' (case-insensitive)",
     db.tasks.find({"title": {"$regex": "^write", "$options": "i"}}, {"_id": 0, "title": 1}))

# Referenced pattern: resolve assignee manually (second query)
task = db.tasks.find_one({"_id": 102})
show("Referenced lookup: task 102 assignee", db.users.find_one({"_id": task["assignee_id"]}, {"_id": 0, "name": 1}))

# ===== UPDATE =====
r = db.tasks.update_one({"_id": 103}, {"$set": {"status": "IN_PROGRESS"}, "$inc": {"estimate_hours": 2}})
print(f"\nupdate_one: matched={r.matched_count}, modified={r.modified_count}")
r = db.tasks.update_one({"_id": 103}, {"$push": {"checklist": {"item": "GET /tasks", "done": False}},
                        "$addToSet": {"tags": "fastapi"}})
print(f"update_one ($push/$addToSet): modified={r.modified_count}")
r = db.tasks.update_many({"project": "Task Hub Frontend"}, {"$set": {"sprint": 4}})

```

```

print(f"update_many: matched={r.matched_count}, modified={r.modified_count}")
r = db.tasks.update_one({"_id": 102, "checklist.item": "Indexes"}, {"$set": {"checklist.$done": True}})
print(f"positional $ update of an embedded item: modified={r.modified_count}")
r = db.tasks.replace_one({"_id": 105}, {"title": "Write release notes v1.0", "project": "Task Hub Frontend",
    "status": "TODO", "priority": 3, "tags": ["docs"]})
print(f"replace_one: modified={r.modified_count}")
r = db.tasks.update_one({"_id": 999}, {"$set": {"title": "Created by upsert", "status": "TODO"}}, upsert=True)
print(f"upsert: upserted_id={r.upserted_id}")
show("task 103 after updates", db.tasks.find_one({"_id": 103}, {"_id": 0, "status": 1, "estimate_hours": 1,
    "tags": 1, "checklist": 1}))
show("task 102 checklist after positional update", db.tasks.find_one({"_id": 102}, {"_id": 0, "checklist": 1}))

# ===== DELETE =====
r = db.tasks.delete_one({"_id": 999})
print(f"\ndelete_one: deleted={r.deleted_count}")
r = db.tasks.delete_many({"status": "DONE"})
print(f"delete_many (status DONE): deleted={r.deleted_count}")
print("Remaining task count:", db.tasks.count_documents({}))
print("Status of remaining tasks:", sorted(db.tasks.distinct("status")))
client.close()

```

crud_mongoose.js

```

// Skillling 5 (Part B): MongoDB CRUD with Mongoose (Node.js) - Task Hub
// Run: npm install mongoose && node crud_mongoose.js
const mongoose = require("mongoose");

const MONGO_URI = process.env.MONGO_URI || "mongodb://127.0.0.1:27017/taskhub_mongoose";

// Referenced model
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true, lowercase: true },
});

// Embedded sub-document schema
const commentSchema = new mongoose.Schema(
  { author: String, text: { type: String, required: true }, at: { type: Date, default: Date.now } },
  { _id: false }
);

const taskSchema = new mongoose.Schema(
  {
    title: { type: String, required: [true, "title is required"], trim: true },
    status: { type: String, enum: ["TODO", "IN_PROGRESS", "DONE"], default: "TODO" },
    priority: { type: Number, min: 1, max: 3, default: 2 },
    assignee: { type: mongoose.Schema.Types.ObjectId, ref: "User" }, // referenced
    tags: [String],
    comments: [commentSchema], // embedded
  },
  { timestamps: true }
);

const User = mongoose.model("User", userSchema);
const Task = mongoose.model("Task", taskSchema);

async function main() {
  await mongoose.connect(MONGO_URI);
  await mongoose.connection.dropDatabase();
  await User.init(); // build the unique index on email
  await Task.init();

  // ----- CREATE -----
  const [habeeb, ananya] = await User.create([
    { name: "Habeeb", email: "Habeeb@TaskHub.dev" },
    { name: "Ananya Rao", email: "ananya@taskhub.dev" },
  ]);
  console.log("Users created:", habeeb.email, "|", ananya.email);

  await Task.insertMany([
    { title: "Design ER diagram", status: "DONE", priority: 1, assignee: habeeb._id, tags: ["db"] },
    { title: "Write FastAPI CRUD", priority: 2, assignee: habeeb._id, tags: ["api", "python"] },
    { title: "Build dashboard page", status: "IN_PROGRESS", priority: 2, assignee: ananya._id, tags: ["ui"] },
    { title: "Write release notes", priority: 3, tags: ["docs"] },
  ]);
  console.log("Tasks inserted:", await Task.countDocuments());

  // Validation errors raised by the schema
  try {

```

```

    await Task.create({ status: "BLOCKED", priority: 7 });
  } catch (err) {
    console.log("Validation failed:", Object.keys(err.errors).sort().join(", "));
  }

  // ----- READ -----
  const highOrMedium = await Task.find({ priority: { $lte: 2 } }).select("title priority -_id").sort({
    priority: 1, title: 1 }).lean();
  console.log("\npriority <= 2:", highOrMedium);

  const open = await Task.find({ status: { $in: ["TODO", "IN_PROGRESS"] }, tags: { $exists: true } })
    .select("title status -_id").sort({ title: 1 }).lean();
  console.log("\nopen tasks ($in):", open);

  const writeTasks = await Task.find({ title: /^write/i }).select("title -_id").lean();
  console.log("\nregex /^write/i:", writeTasks);

  const unassigned = await Task.find({ $or: [{ assignee: null }, { assignee: { $exists: false } }] })
    .select("title -_id").lean();
  console.log("\nunassigned ($or):", unassigned);

  // populate(): resolves the referenced user (Mongoose runs a second query)
  const withUser = await Task.findOne({ title: "Build dashboard page" })
    .populate("assignee", "name email -_id").select("title assignee -_id").lean();
  console.log("\npopulate:", withUser);

  // ----- UPDATE -----
  const upd = await Task.updateOne(
    { title: "Write FastAPI CRUD" },
    { $set: { status: "IN_PROGRESS" }, $push: { comments: { author: "Habeeb", text: "Started GET /tasks" } } }
  );
  console.log("\nupdateOne:", { matched: upd.matchedCount, modified: upd.modifiedCount });

  const updMany = await Task.updateMany({ priority: 3 }, { $set: { priority: 2 } });
  console.log("updateMany:", { matched: updMany.matchedCount, modified: updMany.modifiedCount });

  const updated = await Task.findOneAndUpdate(
    { title: "Build dashboard page" },
    { $addToSet: { tags: "react" } },
    { returnDocument: "after", runValidators: true }
  ).lean();
  console.log("findOneAndUpdate (returnDocument: after):", { title: updated.title, tags: updated.tags });

  const crud = await Task.findOne({ title: "Write FastAPI CRUD" }).select("status comments.author comments.text -_id").lean();
  console.log("after $push of embedded comment:", JSON.stringify(crud));

  // ----- DELETE -----
  const del = await Task.deleteOne({ status: "DONE" });
  console.log("\ndeleteOne:", { deleted: del.deletedCount });
  const delMany = await Task.deleteMany({ tags: "docs" });
  console.log("deleteMany:", { deleted: delMany.deletedCount });
  console.log("Remaining tasks:", (await Task.find().select("title -_id").sort({ title: 1 }).lean()).map((t) => t.title));

  await mongoose.disconnect();
}

main().catch(async (err) => {
  console.error(err);
  await mongoose.disconnect();
  process.exit(1);
});

```

Output

python crud_pymongo.py

```

Inserted users: [1, 2, 3]
insert_one -> 101
insert_many -> [102, 103, 104, 105]
Duplicate email rejected by the unique index

--- find_one by _id ---
{'_id': 101,
 'title': 'Design ER diagram',
 'checklist': [{'item': 'List entities', 'done': True},
               {'item': 'Add cardinalities', 'done': True}]}

```

```

--- find all tasks of 'Task Hub Backend' (projection) ---
{'title': 'Design ER diagram', 'status': 'DONE'}
{'title': 'Create PostgreSQL schema', 'status': 'IN_PROGRESS'}
{'title': 'Write FastAPI CRUD', 'status': 'TODO'}

--- $gte: estimate_hours >= 9 ---
{'title': 'Write FastAPI CRUD', 'estimate_hours': 12}
{'title': 'Build dashboard page', 'estimate_hours': 9}

--- $in: status in TODO / IN_PROGRESS and $ne: priority != 1 ---
{'title': 'Write FastAPI CRUD', 'status': 'TODO', 'priority': 2}
{'title': 'Build dashboard page', 'status': 'IN_PROGRESS', 'priority': 2}
{'title': 'Write release notes', 'status': 'TODO', 'priority': 3}

--- $or: priority 1 OR tagged 'ui' ---
{'title': 'Design ER diagram'}
{'title': 'Create PostgreSQL schema'}
{'title': 'Build dashboard page'}

--- $and + $not: Backend tasks whose status is not DONE ---
{'title': 'Create PostgreSQL schema', 'status': 'IN_PROGRESS'}
{'title': 'Write FastAPI CRUD', 'status': 'TODO'}

--- $nor: neither DONE nor priority 3 ---
{'title': 'Create PostgreSQL schema'}
{'title': 'Write FastAPI CRUD'}
{'title': 'Build dashboard page'}

--- $all: tagged both 'db' and 'sql' ---
{'title': 'Create PostgreSQL schema'}

--- $size: exactly 2 checklist items ---
{'title': 'Design ER diagram'}
{'title': 'Create PostgreSQL schema'}

--- $elemMatch: has an unfinished checklist item ---
{'title': 'Create PostgreSQL schema'}
{'title': 'Build dashboard page'}

--- $exists: tasks without estimate_hours ---
{'title': 'Write release notes'}

--- $type: assignee_id is null (unassigned) ---
{'title': 'Write release notes'}

--- $regex: title starts with 'Write' (case-insensitive) ---
{'title': 'Write FastAPI CRUD'}
{'title': 'Write release notes'}

--- Referenced lookup: task 102 assignee ---
{'name': 'Rahul Verma'}

update_one: matched=1, modified=1
update_one ($push/$addToSet): modified=1
update_many: matched=2, modified=2
positional $ update of an embedded item: modified=1
replace_one: modified=1
upsert: upserted_id=999

--- task 103 after updates ---
{'status': 'IN_PROGRESS',
 'estimate_hours': 14,
 'tags': ['api', 'python', 'fastapi'],
 'checklist': [{'item': 'GET /tasks', 'done': False}]}

--- task 102 checklist after positional update ---
{'checklist': [{'item': 'DDL', 'done': True},
               {'item': 'Indexes', 'done': True}]}

delete_one: deleted=1
delete_many (status DONE): deleted=1
Remaining task count: 4
Status of remaining tasks: ['IN_PROGRESS', 'TODO']

```

node crud_mongoose.js

```

Users created: habeeb@taskhub.dev | ananya@taskhub.dev
Tasks inserted: 4
Validation failed: priority, status, title

```

```

priority <= 2: [
  { title: 'Design ER diagram', priority: 1 },
  { title: 'Build dashboard page', priority: 2 },
  { title: 'Write FastAPI CRUD', priority: 2 }
]

open tasks ($in): [
  { title: 'Build dashboard page', status: 'IN_PROGRESS' },
  { title: 'Write FastAPI CRUD', status: 'TODO' },
  { title: 'Write release notes', status: 'TODO' }
]

regex /^write/i: [ { title: 'Write FastAPI CRUD' }, { title: 'Write release notes' } ]

unassigned ($or): [ { title: 'Write release notes' } ]

populate: {
  title: 'Build dashboard page',
  assignee: { name: 'Ananya Rao', email: 'ananya@taskhub.dev' }
}

updateOne: { matched: 1, modified: 1 }
updateMany: { matched: 1, modified: 1 }
findOneAndUpdate (returnDocument: after): { title: 'Build dashboard page', tags: [ 'ui', 'react' ] }
after $push of embedded comment: {"status":"IN_PROGRESS","comments":[{"author":"Habeeb","text":"Started GET /tasks"}]}

deleteOne: { deleted: 1 }
deleteMany: { deleted: 1 }
Remaining tasks: [ 'Build dashboard page', 'Write FastAPI CRUD' ]

```

Result

Task documents embedded their checklist and comments (read together with the task) and referenced users through assignee_id (shared data), resolved with a second query in PyMongo and with populate() in Mongoose. All CRUD operations (insert_one/insert_many, find with projection, update_one/update_many with \$set, \$inc, \$push, \$addToSet and positional \$, replace_one, upsert, delete_one/delete_many) and every operator family returned the expected documents, and the unique index and Mongoose schema validation rejected invalid data.